

INWK6312 - Lab 5

Source of Truth, Real Traffic Telemetry, and a CI/CD Pipeline

Contents

Introduction	3
Lab Objectives	3
Lab Environment and Preparation	3
Part A: NetBox as Source of Truth	4
Task 0: Environment Check and NetBox Deployment	4
Task 1: Populating NetBox with the Existing Topology	4
Task 2: Querying NetBox over REST and GraphQL	8
Part B: Extending the Topology from the Source of Truth	9
Task 3: Adding Traffic Generating Hosts	9
Task 4: Modeling the New Pieces in NetBox	10
Task 5: Pushing Addressing from NetBox with Ansible	10
Part C: Traffic and Telemetry	13
Task 6: Generating Traffic with iperf3	13
Task 7: Watching Real Telemetry	13
Part D: A CI/CD Pipeline with Ansible and Batfish	15
Task 8: Ansible Based Verification	15
Task 9: Configuring and Verifying NTP with Batfish Checks for cEOS	15
Task 10: Wrapping It in GitHub Actions	18
Task 11: Committing Your Work	20
Clean Up	21
Submission	21
Appendix: Command Summary	22
NetBox	22
Ansible	22
Batfish	22
iperf3	22
Default Credentials	22

Appendix: Node Configuration	23
ceos1	23
ceos2	23
srl1	24

Introduction

This lab covers several topics, including NetBox as a source of truth, receiving telemetry, implementing a CI/CD pipeline, and verification with both Ansible and Batfish.

You will deploy a NetBox instance and populate it with a script. You will extend the network topology used throughout all labs with two hosts to generate traffic through the network and watch it via gNMI. You will also build a small CI/CD pipeline utilizing Ansible and Batfish for doing configuration state verification wrapped in a GitHub Actions workflow that runs automatically when you push.

Lab Objectives

By the end of this lab, you will be able to:

1. Deploy NetBox and populate it programmatically using its Python SDK, rather than through the web interface
2. Query the same data through both a REST and a GraphQL API and compare the two
3. Extend an existing Containerlab topology with new nodes and reconfigure it without losing what was already there
4. Use Ansible to pull configuration data from an external source of truth instead of a hardcoded template variable
5. Generate real network traffic with iperf3 and observe it with a gNMI Subscribe script adapted from a previous lab
6. Write Ansible based verification tasks, register and assert, against live configuration state
7. Write a narrow Batfish check against a single vendor's configuration and understand why it couldn't cover the whole network
8. Wire configuration checks into a GitHub Actions workflow that runs automatically on push

Lab Environment and Preparation

You will need:

- Your network topology
- Your GitHub Classroom repository
- Docker and Docker Compose, already available on your VM

Addressing used in this lab, extending what previous labs already established:

Segment	Node : Interface	Address
ceos1 to host1	ceos1 : Ethernet3	10.0.100.1/24
ceos1 to host1	host1 : eth1	10.0.100.2/24
ceos2 to host2	ceos2 : Ethernet3	10.0.200.1/24
ceos2 to host2	host2 : eth1	10.0.200.2/24

Part A: NetBox as Source of Truth

Task 0: Environment Check and NetBox Deployment

Objective: Deploy NetBox with minimal manual setup to get an API token, everything after that happens through the API.

1. NetBox Docker image is pre-installed, launch it via `docker compose`.

```
cd ~/netbox-docker
docker compose up -d
```

Monitor the output. All volumes and containers should be created. Docker compose could timeout and report “unhealthy” for container `netbox-1`. Ignore that for now.

2. Watch the logs for few minutes until it settles.

```
docker compose logs -f netbox
```

Press Ctrl+C once you see it serving requests.

3. You will need an API token. This token has been created for you in the `docker-compose.override.yml` file.

```
cat docker-compose.override.yml
```

4. Save the token where your scripts can find it using the format “nbt_..”.

```
cd ~/labs
nano .env
```

```
NETBOX_TOKEN=nbt_<SUPERUSER_API_KEY>.<SUPERUSER_API_TOKEN>
NETBOX_URL=http://localhost:8000
```

5. Create this lab’s folder structure

```
mkdir -p lab5/scripts lab5/ansible lab5/checks
```

6. Activate the virtual environment install what this lab needs.

```
source .velab/bin/activate
pip install pynetbox python-dotenv
pip freeze > requirements.txt
```

Questions and Deliverables

1. Provide the output of docker compose logs once NetBox is up, showing it accepting connections.

Task 1: Populating NetBox with the Existing Topology

Objective: model the network nodes in NetBox using Python’s `pynetbox`.

1. Create the data file below. Two nodes are included, you need to add the third node, `sr11`, following the same pattern.

```
cd ~/labs/lab5
nano scripts/netbox_data.yaml
```

```
site:
  name: INWK6312 Lab
  slug: inwk6312-lab
manufacturers:
  - name: Arista
    slug: arista
  - name: Nokia
    slug: nokia
device_types:
  - model: cEOS
    slug: ceos
    manufacturer: arista
  - model: SR Linux
    slug: sr-linux
    manufacturer: nokia
roles:
  - name: Router
    slug: router
    color: "2196f3"

# IP Addresses:
#   ceos1 Ethernet1 (10.0.12.1/30) <-> ceos2 Ethernet1 (10.0.12.2/30)
#   ceos2 Ethernet2 (10.0.23.1/30) <-> srl1 ethernet-1/1.0 (10.0.23.2/30)
#   srl1 ethernet-1/2.0 (10.0.13.1/30) <-> ceos1 Ethernet2 (10.0.13.2/30)
devices:
  - name: ceos1
    device_type: ceos
    role: router
    interfaces:
      - name: Ethernet1
        address: 10.0.12.1/30
      - name: Ethernet2
        address: 10.0.13.2/30
  - name: ceos2
    device_type: ceos
    role: router
    interfaces:
      - name: Ethernet1
        address: 10.0.12.2/30
      - name: Ethernet2
        address: 10.0.23.1/30

# TODO: add srl1, following the same pattern as ceos1 and ceos2 above.
# device_type is sr-linux and its two interfaces are named
# ethernet-1/1.0 and ethernet-1/2.0. Addresses are in the
```

```
# table in the comment above.
```

2. Create the Python script. The script reads `netbox_data.yaml` and creates whatever it describes.

```
nano scripts/populate_netbox.py
```

```
import os
import sys
import yaml
import pynetbox
from dotenv import load_dotenv

load_dotenv()
nb = pynetbox.api(os.environ["NETBOX_URL"],
    ↪ token=os.environ["NETBOX_TOKEN"])

def get_or_create(endpoint, filters, **create_fields):
    """
    Return the existing object matching filters, or create it if it doesn't
    ↪ exist yet.
    Safe to run this whole script more than once.
    """

    existing = endpoint.get(**filters)
    if existing:
        return existing
    return endpoint.create(**{**filters, **create_fields})

def main(data):

    site = get_or_create(
        nb.dcim.sites,
        {"slug": data["site"]["slug"]},
        name=data["site"]["name"],
        status="active",
    )

    manufacturers = {}
    for m in data["manufacturers"]:
        manufacturers[m["slug"]] = get_or_create(
            nb.dcim.manufacturers, {"slug": m["slug"]}, name=m["name"]
        )

    device_types = {}
    for dt in data["device_types"]:
        device_types[dt["slug"]] = get_or_create(
            nb.dcim.device_types,
```

```

        {"slug": dt["slug"]},
        model=dt["model"],
        manufacturer=manufacturers[dt["manufacturer"]].id,
    )

device_roles = {}
for rl in data["roles"]:
    device_roles[rl["slug"]] = get_or_create(
        nb.dcim.device_roles,
        {"slug": rl["slug"]},
        name=rl["name"],
        color=rl["color"],
    )

for dev in data["devices"]:
    device = get_or_create(
        nb.dcim.devices,
        {"name": dev["name"], "site": site.slug},
        site=site.id,
        device_type=device_types[dev["device_type"]].id,
        role=device_roles[dev["role"]].id,
        status="active",
    )

    for iface in dev["interfaces"]:
        interface = get_or_create(
            nb.dcim.interfaces,
            {"device_id": device.id, "name": iface["name"]},
            device=device.id,
            type="other",
        )

        if "address" in iface:
            get_or_create(
                nb.ipam.ip_addresses,
                {"address": iface["address"]},
                assigned_object_type="dcim.interface",
                assigned_object_id=interface.id,
            )

if __name__ == "__main__":

    if len(sys.argv) < 2:
        print("Error: Please provide a filename.")
        sys.exit(1)

    filename = sys.argv[1]

```

```

with open(filename) as f:
    data = yaml.safe_load(f)

main(data)

print(
    f"NetBox populated from netbox_data.yaml: {len(data['devices'])}"
    ↪   device(s) processed"
)

```

3. Run the script.

```
python3 scripts/populate_netbox.py scripts/netbox_data.yaml
```

4. Look at the result in the web UI and confirm that devices, interfaces, and addresses are there. Spend time looking around and discovering NetBox, you won't be using the UI in this lab except for checking.

Questions and Deliverables

1. Provide your completed `netbox_data.yaml`.
2. Provide a screenshot or the text content of one device's interface list from the web UI, confirming step 5.

Task 2: Querying NetBox over REST and GraphQL

Objective: retrieve the same data two different ways, and see why NetBox offers both.

1. Query `ceos1`'s interfaces over the REST API.

```

curl -s -H "Authorization: Token $(grep NETBOX_TOKEN ~/labs/.env | cut -d=
↪   -f2)" \
    "http://localhost:8000/api/dcim/interfaces/?device=ceos1" | python3 -m
↪   json.tool

```

2. Query the same data over GraphQL, asking only for the fields you actually want, name and the addresses assigned to each interface.

```

curl -s -H "Authorization: Token $(grep NETBOX_TOKEN ~/labs/.env | cut -d=
↪   -f2)" \
    -H "Content-Type: application/json" \
    -d '{"query": "{ interface_list(filters: {device: {name: {exact:
↪   \"ceos1\"}}}) { name ip_addresses { address } } }"}' \
    "http://localhost:8000/graphql/" | python3 -m json.tool

```

Questions and Deliverables

1. Provide the output of both queries.

Part B: Extending the Topology from the Source of Truth

Task 3: Adding Traffic Generating Hosts

Objective: extend the network topology file by adding two hosts that connect to the cEOS nodes.

1. Edit the network topology file.

```
nano ~/labs/topology/lab-net.clab.yml
```

Add two new nodes and two new links, alongside the three that are already there.

```
topology:
  nodes:
    # Add these nodes below the original nodes
    host1:
      kind: linux
      image: nicolaka/netshoot:latest
      exec:
        - ip link set dev eth1 mtu 1500
        - ip addr add 10.0.100.2/24 dev eth1
        - ip route replace 0/0 via 10.0.100.1
    host2:
      kind: linux
      image: nicolaka/netshoot:latest
      exec:
        - ip link set dev eth1 mtu 1500
        - ip addr add 10.0.200.2/24 dev eth1
        - ip route replace 0/0 via 10.0.200.1

  links:
    # Add these links below the original links
    - endpoints: ["ceos1:eth3", "host1:eth1"]
    - endpoints: ["ceos2:eth3", "host2:eth1"]
```

The `exec` commands in the topology file set host's default interface MTU to 1500, the IP address of the interface and the default gateway.

2. Deploy the topology.

```
sudo containerlab deploy -t ~/labs/topology/lab-net.clab.yml
```

3. The nodes should retain their past configuration, but if the configuration doesn't come back automatically, then you can apply the configuration list in the Appendix below.
4. You can quickly check if the configuration is correct and if routing works properly:

```
docker exec ceos1 ip route
docker exec ceos1 traceroute 10.0.23.2
```

Note: you can execute the above commands directly from Docker because cEOS runs on Linux and implements the Linux IP command.

Questions and Deliverables

1. Provide the output of `containerlab inspect -t ~/labs/topology/lab-net.clab.yml`, showing all five nodes running.

Task 4: Modeling the New Pieces in NetBox

Objective: extend NetBox data to cover the added nodes, keeping the source of truth in sync with the actual topology.

1. Extend `netbox_data.yaml` by adding entries for `host1` and `host2`, plus the two new router interfaces, `Ethernet3` on both `ceos1` and `ceos2`.

```
cd ~/labs/lab5
nano scripts/netbox_data.yaml
```

Add a manufacturer, a role, and a device type for the hosts:

```
manufacturers:
  - name: Nicolaka
    slug: nicolaka
device_types:
  - model: Generic
    slug: generic
roles:
  - name: Host
    slug: host
    color: "ffa500"
```

2. Add the two host devices, `host1` and `host2`, with one interface each, and IP addresses matching the addressing table in this lab's introduction.
3. Add interface `Ethernet3` to nodes `ceos1` and `ceos2` with IP addresses matching the addressing table in this lab's introduction.
4. Save the YAML file then run `populate_netbox.py` again.

```
python3 scripts/populate_netbox.py scripts/netbox_data.yaml
```

Questions and Deliverables

1. Provide a copy of the extended `netbox_data.yaml`.
2. Query the REST API for `host1`'s IP address, the same way you did in Task 2, and provide the output.

Task 5: Pushing Addressing from NetBox with Ansible

Objective: use the source of truth to drive automation.

1. Create an Ansible playbook that queries NetBox for each host's `Ethernet3` address rather than having that address typed into a template variable anywhere.

```
cd ~/labs/lab5
nano ansible/configure_hosts_interface.yml
```

```
---
- name: Configure host facing interface from NetBox
  hosts: ceos
  gather_facts: false
  vars:
    netbox_url: "http://localhost:8000"
    netbox_token: "{{ lookup('ansible.builtin.ini', 'NETBOX_TOKEN',
↪ type='properties', file='~/labs/.env') }}"
  tasks:
    - name: Look up this device's Ethernet 3 address in NetBox
      ansible.builtin.uri:
        url: "{{ netbox_url }}/api/ipam/ip-addresses/?device={{
↪ inventory_hostname }}&interface=Ethernet3"
        headers:
          Authorization: "Token {{ netbox_token }}"
          return_content: true
        register: netbox_response

    - name: Extract the address from the response
      ansible.builtin.set_fact:
        host_facing_address: "{{ netbox_response.json.results[0].address }}"

    - name: Debug the extracted address
      ansible.builtin.debug:
        msg: "Extracted address: {{ host_facing_address }}"

    - name: Apply the address and join OSPF area 0.0.0.0
      arista.eos.eos_config:
        lines:
          - interface Ethernet 3
          - no switchport
          - ip address {{ host_facing_address }}
          - ip ospf area 0.0.0.0
          - no shutdown

    - name: Update OSPF configuration
      arista.eos.eos_config:
        lines:
          - no passive-interface Ethernet 3
        parents: router ospf 100
```

2. Use the Ansible inventory from Lab4 to run the playbook.

```
ansible-playbook -i ~/labs/lab4/ansible/inventory.yml
↪ ansible/configure_hosts_interface.yml
```

-
3. Verify on the network node.

```
ansible -i ~/labs/lab4/ansible/inventory.yml ceos -m arista.eos.eos_command
↳ -a "commands='show ip interface brief'"
ansible -i ~/labs/lab4/ansible/inventory.yml ceos -m arista.eos.eos_command
↳ -a "commands='show ip route ospf'"
```

4. Verify from both hosts, using their Ethernet1 interfaces addresses instead of their hostnames. `host1` and `host2` resolve through `/etc/hosts` to their management addresses, not their Ethernet1 ones, so pinging by hostname here would test the wrong path entirely.

```
docker exec host1 traceroute 10.0.200.2
docker exec host2 ping -c 5 10.0.100.2
```

Questions and Deliverables

1. Provide the output of step 3, and the ping output from step 4.
2. If you changed `host1`'s address in NetBox and reran this playbook, with nothing else touched, what would you expect Ansible to report, ok or changed? What does that tell you about where this playbook's actual source of truth lives?

Part C: Traffic and Telemetry

Task 6: Generating Traffic with iperf3

Objective: create sustained traffic between the two hosts.

1. Start an iperf3 server on host2.

```
docker exec host2 iperf3 -s -D
```

2. Run a client test from host1 for 30 seconds at 1Mbps. You must use the IP address of the destination rather than its name. Why?

```
docker exec host1 iperf3 -c 10.0.200.2 -t 30 -b 1M
```

Questions and Deliverables

1. Provide the summary line from the end of the iperf3 output, showing the achieved throughput.

Task 7: Watching Real Telemetry

Objective: reuse gNMI subscribe script from Lab 4 to report telemetry from ceos2.

1. Copy subscribe_sample.py from Lab 4 and change the node to ceos1 and the path.

```
cd ~/labs/lab5/scripts
cp ~/labs/lab4/scripts/subscribe_sample.py subscribe_ceos_traffic.py
nano subscribe_ceos_traffic.py
```

```
import json
from pygnmi.client import gNMIclient, telemetryParser
from devicelib import load_devices

devices = load_devices("gnmi")
ceos1 = devices["ceos1"]

subscribe_request = {
    "subscription": [
        {
            "path":
                ↪ "interfaces/interface[name=Ethernet3]/state/counters/in-octets",
            "mode": "sample",
            "sample_interval": 1000000000,
        }
    ],
    "mode": "stream",
    "encoding": "json",
}

with gNMIclient(**ceos1) as gc:
    telemetry_stream = gc.subscribe(subscribe=subscribe_request)
```

```
for response in telemetry_stream:
    print(json.dumps(telemetryParser(response), indent=2))
```

This watches Ethernet3, ceos1's link to host1 which is generating the traffic towards host2.

2. Start the script, then in a second terminal, run another iperf3 test while it's running.

```
python3 subscribe_ceos_traffic.py
```

In a second terminal:

```
docker exec host2 iperf3 -s -D
docker exec host1 iperf3 -c 10.0.200.2 -t 30 -b 1M
```

3. Watch the counter values between updates, the difference between consecutive samples should track roughly with the throughput iperf3 reported.
4. Stop the script with Ctrl+C once the test finishes.

Questions and Deliverables

1. Provide about a minute of output from step 2, spanning before, during, and after the iperf3 run.
2. Pick two consecutive samples taken during the iperf3 run. The difference in out-octets between them, divided by the 10 second sample interval, gives you an average byte rate. How does that compare to the throughput iperf3 itself reported in Task 6?

Part D: A CI/CD Pipeline with Ansible and Batfish

Task 8: Ansible Based Verification

Objective: write verification tasks that check live state against what you expect, extending the register and assert pattern from Lab 4's compliance check.

1. Create the playbook.

```
cd ~/labs/lab5
nano ansible/verify_network.yml

---
- name: Verify OSPF and reachability
  hosts: ceos
  gather_facts: false
  tasks:
    - name: Check OSPF neighbor state
      arista.eos.eos_command:
        commands:
          - show ip ospf neighbor
      register: ospf_output

    - name: Assert both expected neighbors are FULL
      ansible.builtin.assert:
        that:
          - "'FULL' in ospf_output.stdout[0]"
        fail_msg: "No FULL OSPF neighbors found"
        success_msg: "OSPF adjacency confirmed"
```

2. Run it.

```
ansible-playbook -i ~/labs/lab4/ansible/inventory.yml
↪ ansible/verify_network.yml
```

Questions and Deliverables

1. Provide the output of step 2.
2. This check only confirms the word FULL appears somewhere in the output, it doesn't count how many neighbors, or check which specific ones. Extend the assert condition to also fail if fewer than 2 neighbors show FULL, and provide your modified task.

Task 9: Configuring and Verifying NTP with Batfish Checks for cEOS

Objective: add NTP configuration to cEOS nodes. Before touching the production network, use Ansible to back up the config, add NTP configuration to the configuration files, then use Batfish to test the correctness of the NTP configuration and reachability to the NTP server.

Note: Batfish has no support for Nokia SR Linux at the moment, so this Task only covers `ceos1` and `ceos2`.

1. Start Batfish, its Docker image has already been pulled into your machine.

```
cd ~/labs
docker run -d --name batfish -p 9997:9997 -p 9996:9996 batfish/allinone
pip install pybatfish
pip freeze > requirements.txt
```

2. Pull ceos1 and ceos2's running configs into a snapshot folder.

```
mkdir -p ~/labs/lab5/checks/ceos/configs
```

```
nano ~/labs/lab5/ansible/backup_configs.yml
```

```
---
- name: Back up running configuration
  hosts: ceos
  gather_facts: false
  tasks:
    - name: Retrieve running config
      arista.eos.eos_command:
        commands:
          - show running-config
      register: running_config

    - name: Save to snapshot folder
      ansible.builtin.copy:
        content: "{{ running_config.stdout[0] }}"
        dest: "/home/student/labs/lab5/checks/ceos/configs/{{
        ↪ inventory_hostname }}.cfg"
```

```
ansible-playbook -i ~/labs/lab4/ansible/inventory.yml
↪ ~/labs/lab5/ansible/backup_configs.yml
```

3. Edit ceos1's configuration file to add NTP near the end of the file, then save.

```
nano ~/labs/lab5/checks/ceos/configs/ceos1.cfg
```

```
!
ntp server 172.20.20.1
end
```

4. Repeat for ceos2, but use a different (incorrect) server.

```
!
ntp server 1.1.1.1
end
```

5. Write the check. This confirms NTP is configured, the server IP address is among an approved list of servers, and the server is reachable.


```
nano ~/labs/lab5/checks/batfish_ntp_check.py
```

```
from pybatfish.client.session import Session
from pybatfish.datamodel.flow import HeaderConstraints, PathConstraints

bf = Session(host="localhost")
bf.set_network("lab5")
bf.init_snapshot(
    "ceos",
    name="snapshot",
    extra_args={"ignoremanagementinterfaces": False},
    overwrite=True,
)

COL_NAME = "NTP_Servers"

# 1. Fetch configured NTP servers using nodeProperties
node_props = bf.q.nodeProperties(properties=COL_NAME).answer().frame()

# Define the reference set of allowed/expected NTP servers
ref_ntp_servers = set(["172.20.20.1"])

# 2. Find nodes that have no NTP server in common with the reference set
ns_violators = node_props[
    node_props[COL_NAME].apply(lambda x:
↪ len(ref_ntp_servers.intersection(set(x))) == 0)
]

print("--- NTP Policy Violations ---")
if not ns_violators.empty:
    print("FAIL: Nodes with no overlapping reference NTP servers found:")
    print(ns_violators[["Node", COL_NAME]])
    raise SystemExit(1)
else:
    print("PASS: All nodes have at least one valid NTP server from the
↪ reference set.")

# 3. Verify network reachability to the validated NTP server(s)
print("\n--- NTP Server Reachability Check ---")
all_reachable = True

# Retrieve all node names dynamically
nodes_df = bf.q.nodeProperties().answer().frame()

for node in node_props["Node"]:
    for ip in ref_ntp_servers:
        headers = HeaderConstraints(dstIps=ip, ipProtocols=["UDP"],
↪ dstPorts=["123"])
```

```

        r_df = (
            bf.q.reachability(
↪ pathConstraints=PathConstraints(startLocation=f"{node}[Management0]"),
                headers=headers,
                actions="Delivered_to_subnet",
            )
            .answer()
            .frame()
        )

        # Check if any trace reached an ACCEPTED status
        reachable = False
        if not r_df.empty and "Traces" in r_df.columns:
            reachable = any(
                trace.disposition == "DELIVERED_TO_SUBNET" # "ACCEPTED"
                for traces in r_df["Traces"]
                for trace in traces
            )

        if reachable:
            print(f"PASS: Node '{node}' can reach NTP server {ip} on UDP
↪ 123.")
        else:
            print(f"FAIL: Node '{node}' cannot reach NTP server {ip}.")
            all_reachable = False

    if not all_reachable:
        raise SystemExit(1)

```

6. Run it. The check should fail because server 1.1.1.1 is not among the approved servers.

```

cd ~/labs/lab5/checks
python3 batfish_ntp_check.py

```

Questions and Deliverables

1. Provide the output of step 4, showing a check failure due to ceos2's unapproved NTP server.
2. The reachability check confirms Batfish's model routes NTP traffic toward 172.20.20.1's subnet. What could this check miss that only testing against the live network could catch?

Task 10: Wrapping It in GitHub Actions

Objective: run the checks from Task 9 automatically whenever configuration work gets pushed to GitHub.

1. Create the workflow file in your repository.

```

cd ~/labs
mkdir -p ~/labs/.github/workflows
nano ~/labs/.github/workflows/config-check.yml

name: Config Checks

on:
  push:
    paths:
      - "lab5/checks/**"

jobs:
  batfish-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4.2.2

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Start Batfish
        run: docker run -d --name batfish -p 9997:9997 -p 9996:9996
        ↪ batfish/allinone

      - name: Install pybatfish
        run: pip install pybatfish

      - name: Wait for Batfish service
        run: |
          echo "Waiting for Batfish container to accept connections..."
          for i in {1..30}; do
            if curl -s http://localhost:9997/ > /dev/null 2>&1 || nc -z
            ↪ localhost 9997; then
              echo "Batfish is ready after ${i} seconds!"
              exit 0
            fi
            sleep 1
          done
          echo "Timed out waiting for Batfish."
          exit 1

      - name: Run configuration check
        run: python3 batfish_ntp_check.py
        working-directory: lab5/checks

```

2. Commit and push just this file, then confirm it actually caught nothing to run yet, this

workflow only triggers on changes under lab5/checks.

GitHub won't accept a push that adds or modifies a file under `.github/workflows/` using a token that only has repo scope, the scope Lab 1 had you create. It requires a token with the workflow scope specifically. Check your token's scopes on GitHub under Settings, Developer settings, Personal access tokens, and add workflow if it isn't already there, or generate a new token with both scopes and use that instead.

```
cd ~/labs
git add .github/workflows/config-check.yml
git commit -m "Add config check workflow"
git push
```

3. Add the checks folder, then commit and push. `ceos2.cfg` is still carrying the deliberate error from Task 9, so this push should trigger a failing run.

```
git add lab5/checks/
git commit -m "Add checks folder"
git push
```

4. Confirm the Actions tab shows a failed run this time, and that the log names the actual reason, no overlapping reference NTP servers found.
5. Change the server IP to "172.20.20.1" in `ceos2`, then repeat step 3 and confirm that the check passes.

Questions and Deliverables

1. Provide a link to your GitHub Actions run history, or paste the relevant log excerpt, showing one failing run and one passing run.
2. The workflow only triggers on changes under lab5/checks, not on every push to the repository. Why does that scoping matter, given this repository also contains other files?

Task 11: Committing Your Work

Objective: bring the rest of this lab's files into your repository.

1. Move to the root of your repository.

```
cd ~/labs
git status
```

2. Stage and confirm what is about to be committed.

```
git add topology/lab-net.clab.yml lab5 requirements.txt
git status
```

3. Commit and push.

```
git commit -m "Add Lab 5, NetBox source of truth, real traffic telemetry,
↳ and CI/CD pipeline"
git push
```

Questions and Deliverables

1. Provide the output of `git log --oneline -5` after your commit.

Clean Up

Save the network's configuration, then destroy it.

```
sudo containerlab save -t ~/labs/topology/lab-net.clab.yml
sudo containerlab destroy -t ~/labs/topology/lab-net.clab.yml
```

NetBox and Batfish are not part of the saved topology state, stop them separately if you want to reclaim the resources.

```
cd ~/netbox-docker && docker compose down
docker stop batfish && docker rm batfish
```

Deactivate your virtual environment.

```
deactivate
```

Submission

Confirm your repository includes, at minimum, the following, then submit as instructed by your course delivery platform:

- The modified `labs/topology/lab-net.clab.yml`, with `host1` and `host2` added
- `lab5/scripts/populate_netbox.py` and `subscribe_ceos_traffic.py`
- `lab5/ansible/configure_hosts_interface.yml`, `verify_network.yml`, and `backup_configs.yml`
- `lab5/checks/batfish_ntp_check.py`
- `.github/workflows/config-check.yml`
- Your answers to the Questions and Deliverables sections, submitted as your lab report per your instructor's separate instructions

Appendix: Command Summary

NetBox

Command	Usage
<code>docker compose up -d</code>	Start the NetBox stack in the background
<code>docker compose logs -f netbox</code>	Follow NetBox's startup logs
<code>pynetbox.api(url, token=token)</code>	Open an API session from Python
<code>nb.dcim.devices.create(...)</code>	Create a device
<code>nb.ipam.ip_addresses.create(...)</code>	Create and optionally assign an IP address

Ansible

Call	Usage
<code>ansible.builtin.uri</code>	Make an HTTP request, used here to query NetBox's REST API
<code>ansible.builtin.set_fact</code>	Store a value extracted from a previous task's result
<code>until / retries / delay</code>	Retry a task until a condition in its result is met

Batfish

Call	Usage
<code>Session(host="localhost")</code>	Connect to a running Batfish instance
<code>bf.init_snapshot(folder, name=, overwrite=)</code>	Load a folder of device configs as a snapshot
<code>bf.q.interfaceProperties().answer().frame()</code>	Query interface state as a table
<code>bf.q.ospfInterfaceConfiguration().answer().frame()</code>	Query per-interface OSPF configuration as a table

iperf3

Command	Usage
<code>iperf3 -s</code>	Run as a server, listening for a client
<code>iperf3 -c <address> -t <seconds></code>	Run as a client against a server for a fixed duration

Default Credentials

Kind	Username	Password
Arista cEOS	admin	admin
Nokia SR Linux	admin	NokiaSrl1!
NetBox	admin	admin

Appendix: Node Configuration

Copy and paste the configuration below into each network node.

ceos1

```
interface Ethernet1
  no switchport
  ip address 10.0.12.1/30
  ip ospf area 0.0.0.0
!
interface Ethernet2
  no switchport
  ip address 10.0.13.2/30
  ip ospf area 0.0.0.0
!
interface Loopback0
  ip address 10.255.0.1/32
  ip ospf area 0.0.0.0
!
interface Management0
  ip address 172.20.20.2/24
  ipv6 address 3fff:172:20:20::2/64
!
ip routing
!
router ospf 100
  router-id 10.255.0.1
  passive-interface default
  no passive-interface Ethernet1
  no passive-interface Ethernet2
  no passive-interface Loopback0
  max-lsa 12000
!
end
```

ceos2

```
interface Ethernet1
  no switchport
  ip address 10.0.12.2/30
  ip ospf area 0.0.0.0
!
interface Ethernet2
  description Link to srl1
  no switchport
  ip address 10.0.23.1/30
```

```

    ip ospf area 0.0.0.0
!
interface Loopback0
    ip address 10.255.0.2/32
    ip ospf area 0.0.0.0
!
interface Management0
    ip address 172.20.20.4/24
    ipv6 address 3fff:172:20:20::4/64
!
ip routing
!
router ospf 100
    router-id 10.255.0.2
    passive-interface default
    no passive-interface Ethernet1
    no passive-interface Ethernet2
    no passive-interface Loopback0
    max-lsa 12000
!
end

```

sr11

```

set / interface ethernet-1/1 admin-state enable
set / interface ethernet-1/1 subinterface 0 ipv4 admin-state enable
set / interface ethernet-1/1 subinterface 0 ipv4 address 10.0.23.2/30
set / interface ethernet-1/2 admin-state enable
set / interface ethernet-1/2 subinterface 0 ipv4 admin-state enable
set / interface ethernet-1/2 subinterface 0 ipv4 address 10.0.13.1/30

set / interface system0 admin-state enable
set / interface system0 subinterface 0 ipv4 admin-state enable
set / interface system0 subinterface 0 ipv4 address 10.255.0.3/32

set / network-instance default interface ethernet-1/1.0
set / network-instance default interface ethernet-1/2.0
set / network-instance default interface system0.0
set / network-instance default protocols ospf instance 100 admin-state enable
set / network-instance default protocols ospf instance 100 version ospf-v2
set / network-instance default protocols ospf instance 100 router-id 10.255.0.3
set / network-instance default protocols ospf instance 100 area 0.0.0.0 interface
↪ ethernet-1/1.0 admin-state enable
set / network-instance default protocols ospf instance 100 area 0.0.0.0 interface
↪ ethernet-1/2.0 admin-state enable
set / network-instance default protocols ospf instance 100 area 0.0.0.0 interface
↪ system0.0 admin-state enable

```